

CRUCE AI DEVELOPMENT REPORT

Date: August 13, 2026

Repository: D:\Projects\Playground\AI Assisted Projects\Cruce

SUMMARY

Today I added and evolved the Cruce AI from simple legal-action automation into a data-generating, trainable, and playable AI pipeline. The current playable bots are available in the lobby as ai_bot_1 through ai_bot_4. They can be invited to 2-, 3-, or 4-player games, accept invitations automatically, select target scores when chosen, bid, choose trump, play legal cards, and accept rematches.

CURRENT AI COMPONENTS

1. Live playable bot, C++: implemented in src/app/server_main.cpp. This is the AI used by the server when players invite ai_bot_1 through ai_bot_4.
2. Move database: JSONL event records are appended to data/game_events.db. Match summaries are appended to data/match_history.db.
3. Training export: tools/export_training_data.py converts logged events into bid, trump, and card-play datasets under data/ai/.
4. Model code: src/ai/model.py contains a dependency-free sparse perceptron. src/ai/train.py trains models. src/ai/policy.py loads trained models and chooses actions.
5. Model-guided self-play: tools/ai_self_play.py runs games through the normal HTTP API using trained models.

STEPS COMPLETED TODAY

1. Added persistent AI training logs. Every important game action can be saved with actor, request, result, match id, private state before the action, and private state after the action. This makes the game history usable later for supervised learning and reinforcement-learning experiments.
2. Added match-history persistence. Completed matches are summarized with players, winner, target score, final score, and round count in data/match_history.db.
3. Added server self-play. The executable supports commands such as build\Debug\cruce_server_app.exe --self-play 500 --players 2 --target 6 --seed 20260814. This creates valid bot-vs-bot games without opening any client.
4. Generated self-play data. Earlier batches generated 500 games for each player mode with the initial valid-action bot. Today I also completed a stronger 2-player batch: 500 games, 0 failures, 38,381 actions logged. A later 3-player generation was interrupted by the user request for this PDF, so I stopped the remaining generator process.
5. Exported training examples. The exporter maps bid, trump, and card-play actions into separate JSONL datasets. Card examples include legalCardIds as a mask so the model cannot choose impossible cards.
6. Trained baseline models. Models were trained into versioned folders under data/ai/models/, including v1 and v2. These are lightweight sparse linear classifiers that run without third-party Python packages.
7. Added a playable AI path. Built-in bot users ai_bot_1 through ai_bot_4 are always visible as online Bot players. Human users can invite them from the lobby and play complete matches.
8. Improved the live heuristic bot. The previous bot mostly bid 1, chose the strongest-looking suit, led high, and otherwise played low. The improved bot now evaluates hand strength, high cards, marriage announcements, trump potential, target score pressure, current trick value, whether its own team is already winning the trick, and whether a legal card can capture the trick.

9. Improved 4-player team behavior. In team games, the bot now checks ownership, avoids unnecessarily overtaking a partner, and can feed points to a partner who is already winning the trick.
10. Improved announcement awareness. The bot values King-Queen pairs and favors playing one card from the pair when it creates 20 or 40 announcement points.
11. Improved target selection. AI-selected match targets are no longer always 6 points. Bot target selection now varies among 6, 11, and 21.
12. Enriched model features. `src/ai/features.py` now includes trick points, current winner ownership, legal cards that can win, legal cards that feed the same side, announcement opportunities, likely trump candidate, and more state context.

VALIDATION RUN TODAY

- `cmake --build build`: passed after fixing Windows min/max macro collisions.
- `ctest --test-dir build -C Debug --output-on-failure`: 7 of 7 tests passed.
- Stronger 2-player self-play: 500 of 500 games completed, 0 failed, 38,381 actions logged.
- Stopped interrupted self-play process PID 20212 before writing this report.

CURRENT DATA COUNTS

- `data/game_events.db`: 266,922 logged event rows.
- `data/match_history.db`: 2,536 match-history rows.

TRAINED MODEL METRICS AVAILABLE NOW

Version v1:

- Bid model: train 17,659 examples, eval 4,423 examples, eval accuracy 0.9989.
- Trump model: train 5,750 examples, eval 1,442 examples, eval accuracy 0.9452.
- Card model: train 138,000 examples, eval 34,608 examples, eval accuracy 0.7271.

Version v2:

- Bid model: train 17,741 examples, eval 4,439 examples, eval accuracy 0.9991.
- Trump model: train 5,791 examples, eval 1,450 examples, eval accuracy 0.9310.
- Card model: train 138,984 examples, eval 34,800 examples, eval accuracy 0.7595.

IMPORTANT NOTE ABOUT CURRENT STATE

The live playable bots now use the improved C++ heuristic. The already-saved v1 and v2 Python models were trained before the final stronger-heuristic pass in this session. The stronger 2-player self-play data has been appended to the database, but the full stronger 3-player and 4-player batches plus a new v3 retraining pass were interrupted by the request to create this PDF.

RECOMMENDED NEXT COMMANDS

1. Finish stronger self-play data for all modes:
`build\Debug\cruce_server_app.exe --self-play 500 --players 3 --target 6 --seed 20260815`
`build\Debug\cruce_server_app.exe --self-play 500 --players 4 --target 6 --seed 20260816`
2. Export the updated datasets:
`python tools/export_training_data.py --include-real`
3. Train the next model version:
`python src/ai/train.py --data data/ai --models data/ai/models/v3 --epochs 5`
4. Compare model versions:
`python tools/compare_ai_models.py data/ai/models/v2 data/ai/models/v3`

FILES MOST RELEVANT TO THE CURRENT AI

- src/app/server_main.cpp: live bots, self-play simulator, action logging, HTTP app endpoints.
- tools/export_training_data.py: converts move logs to model datasets.
- src/ai/features.py: feature extraction and legal card mask helpers.
- src/ai/model.py: sparse perceptron implementation.
- src/ai/train.py: model training and metrics output.
- src/ai/policy.py: model inference for bid, trump, and card play.
- tools/ai_self_play.py: trained-model self-play through the server API.
- tests/python/ai_bot_smoke.py: verifies that a human can invite and finish a match against a bot.
- README.md: documents AI data generation, training, and usage commands.